

Hindustan College of Science and Technology, Farah, Mathura

Department of Information Technology

FINAL YEAR PROJECT ON

BLOOD GROUP PREDICTION USING IMAGE PROCESSING AND ARTIFICIAL NEURAL
NETWORK

Group No: HCST/IT/2024/03

Project Guide:
Mrs. Deepti Mittal



Group Members:

Ananya Dubey (2100640130008)
Devesh Kumar (2100640130019)
Palak Sharma (2100640130039)

Prerequisite

What is Blood group ?

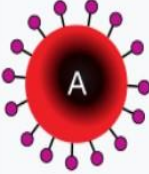
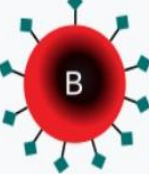
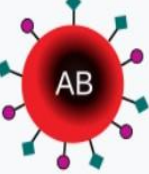
A blood group is a classification of blood, based on the presence and absence of the antibodies and inherited antigenic substances on the surface of red blood cells (RBCs).

What are its types ?

A , B , AB , O

What is Rh blood group system ?

system for classifying blood groups according to the presence or absence of the Rh antigen

	Group A	Group B	Group AB	Group O
Red blood cell type				
Antibodies in plasma	 Anti-B	 Anti-A	None	 Anti-A and Anti-B
Antigens in red blood cell	 A antigen	 B antigen	 A and B antigens	None

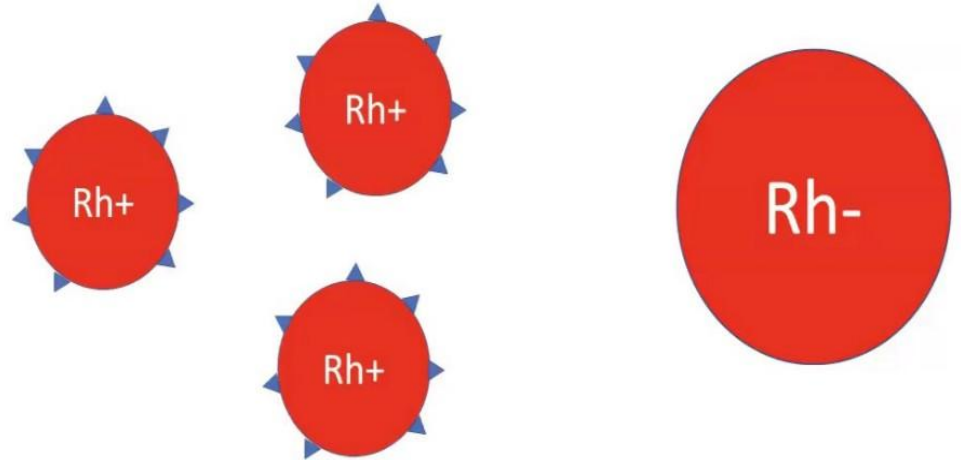
Prerequisite

What is Fingerprint pattern ?

The most reliable and unique feature of human identity is the fingerprint pattern. The fingerprint pattern cannot be changed and remains as is till death of an individual.

What is Rh Antigen ?

the proteins present on outer layer of RBCs is Rh Antigen.



Problem Statement

- Accurate and rapid determination of blood groups is a critical component of medical diagnostics
- Current methods, while accurate, often rely on laboratory-based serological tests that can be time-consuming and require skilled personnel.
- In resource-constrained environments or in situations where rapid diagnosis is essential, such methods may introduce delays in treatment.
- Many chronic diseases remain undiagnosed until advanced stages due to the unavailability of cost effective and convenient early detection tool.
- Fingerprints, despite being a readily available biometric, are underutilized in healthcare for non-invasive diagnosis of blood groups and chronic disease.
- There is a need for reliable and efficient system to bridge the gap between advanced diagnostic methods and accessible healthcare services in underserved regions.

Existing System

- Traditional blood group determination methods require invasive blood sample collection and laboratory analysis , which are time consuming and resource intensive.
- Current blood group detection methods rely on serological test , which involve analyzing blood samples to identify specific antigens present on surface of RBCs.

Agglutination Tests

- Direct Agglutination : Mixing of blood with antibodies to see if clumping (agglutination) occurs , indicating the presence of specific antigen.
- Reverse Typing : Mixing blood plasma with known antigen to detect the presence of antibodies.

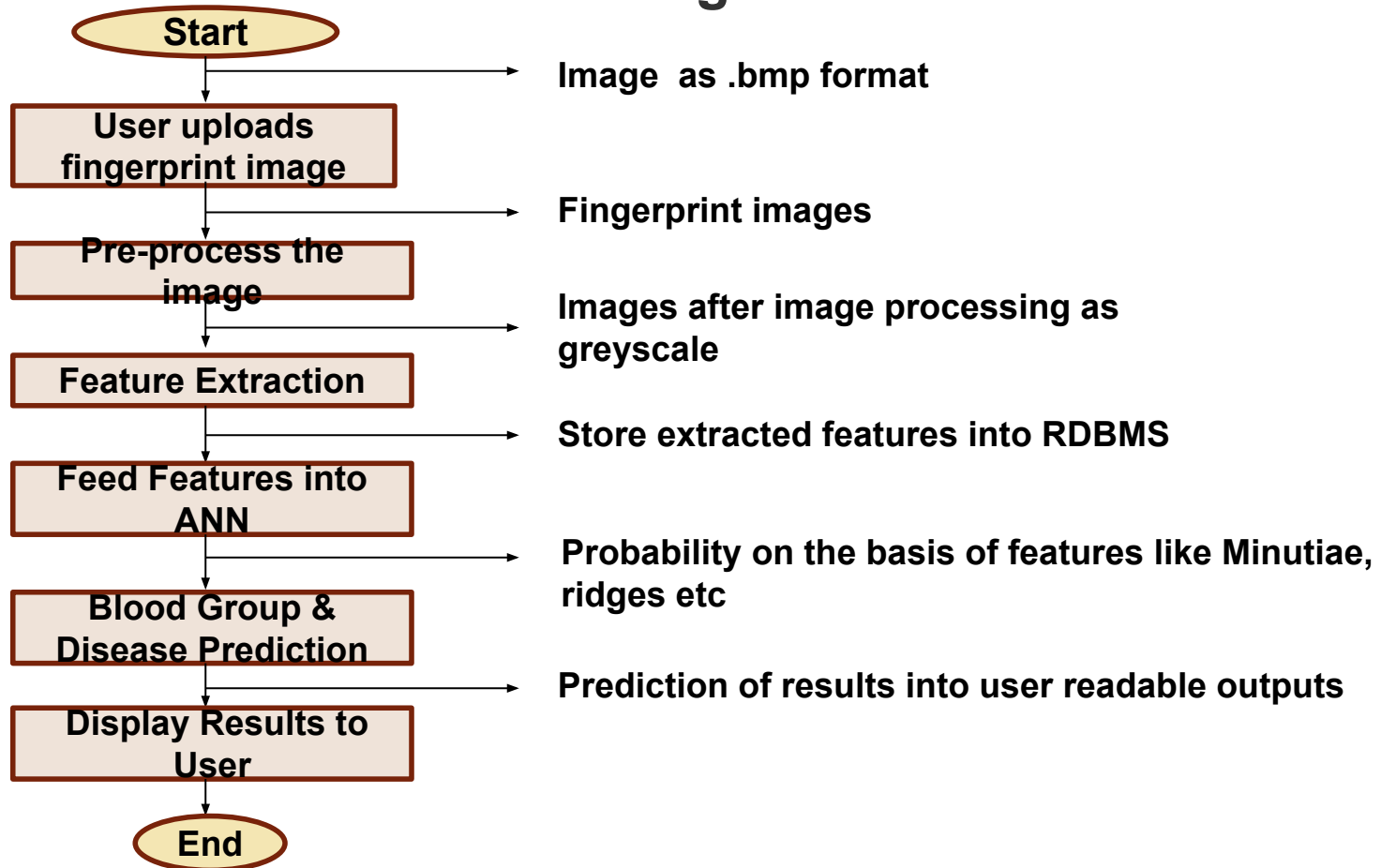
Disadvantages of existing system

- **Invasiveness** : Require Blood samples causing discomfort and potential infection risk.
- **Time consuming** : Require a significant time for Lab test.
- **Resource Intensive** : Require Specialized equipment and trained lab technicians.
- **Cost** : This can be expensive due to need for reagents and maintenance of lab equipments.

Proposed System

- This project aims to develop an automated system for blood group detection using image processing and artificial neural networks (ANN).
- Proposed system for blood group detection uses deep learning to analyze fingerprint images , providing a non invasive , rapid and accessible alternative to traditional methods.
- By collecting a comprehensive dataset of fingerprint images, preprocessing these images through normalization and augmentation and extraction of fingerprint features , the system leverages a Artificial Neural Network (ANN) for training the model.
- The ANN is trained to classify blood groups (A , B, AB, O) with high accuracy , evaluated using metrics as accuracy , precision , recall and F1 – Score.
- This is innovative approach offer significant benefits in terms of speed , cost effectiveness and ease of use making it valuable for resource limited settings.
- No need of blood samples and laboratory equipments for blood group determination.

Work flow diagram



Introduction

- The healthcare industry is continuously evolving to adopt non-invasive, cost-effective, and accessible diagnostic methods to cater to diverse populations.
- Blood group identification is essential for medical procedures like transfusions, but traditional methods are invasive and require specialized infrastructure.
- Chronic diseases significantly contribute to global mortality, highlighting the need for accessible and early diagnostic solutions.
- Fingerprints, as unique and non-invasive biometrics, offer untapped potential for healthcare diagnostics.
- Advances in image processing and machine learning now enable the extraction of meaningful patterns from biometric data, facilitating innovative medical applications.
- This project aims to utilize fingerprint analysis to detect blood groups and predict chronic diseases, providing a cost-effective and accessible diagnostic tool.
- The proposed system addresses healthcare challenges in resource-limited regions by reducing reliance on expensive equipment and invasive testing.

Types Of Fingerprint Patterns

There are three different types of fingerprint patterns:



loop

60 – 65 %



whorl

30 – 35 %

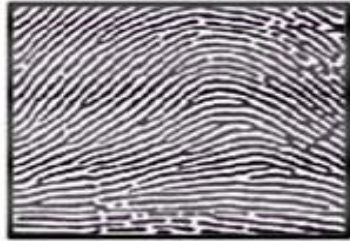


arch

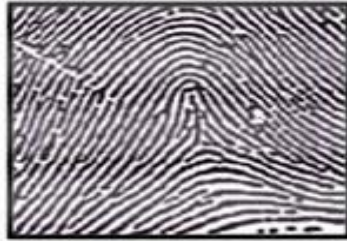
5 %

Arches

Ridges enter on one side & exit on the other side.



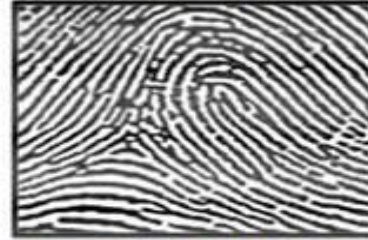
Plain Arch



Tented Arch

Loops

Ridges enter on one side & exit on the same side



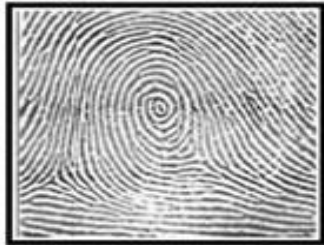
L - Radial Loop
R - Ulnar Loop



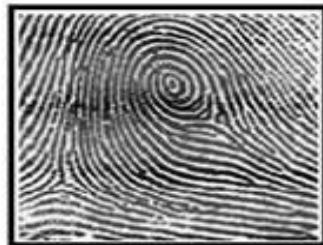
L - Ulnar Loop
R - Radial Loop

Whorls

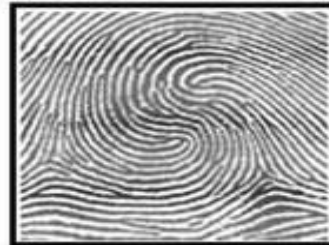
Consists of circles, more than one loop, or a mixture of pattern types



Plain
Whorl



Central
Pocket
Whorl

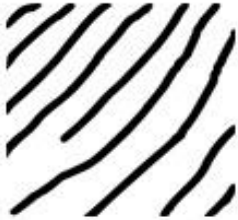


Double
Loop
Whorl



Accidental
Whorl

Fingerprint Minutiae Features



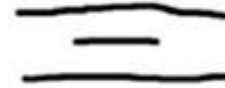
Ridge ending



Bifurcation



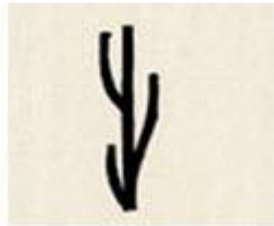
Short Ridge



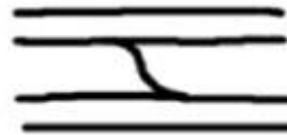
Island



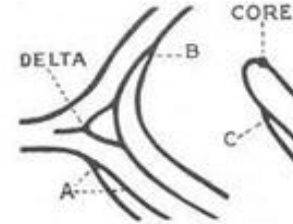
Ridge enclosure



Spur



Bridge



Delta and Core

Feasibility Study

Technical Feasibility

Languages & framework

HTML
CSS
JavaScript } for frontend

Python with
Django → for backend

System Requirements

- ✓ Cloud or Local Server
- ✓ Database management system (MySQL)
- ✓ Internet Connectivity

Image Processing requirements

- ✓ Image processing library } OpenCV
- ✓ Artificial Neural networks (ANN)
└─┬─┘
For blood group detection

Operational Feasibility

Accessibility

- ✓ Designed as web application
- ✓ Accessible on any browser
- ✓ No installation required
- ✓ Simple UI

Integration & Workflow

- ✓ Simple process for uploading fingerprint images and viewing results
- ✓ Operates independently
- ✓ Complement current manual methods of blood & disease testing
- ✓ Allow users to provide feedback
- ✓ Robust pre-processing
- ✓ Workflow: $\begin{matrix} \text{Input} & \xrightarrow{\quad} & \text{Pre-process} & \xrightarrow{\quad} & \text{extract} \\ & & \text{predict} & & \text{output} \end{matrix}$

Scalability

- ✓ Easily upgradable for future features
- ✓ Handles more number of users without slowing down performance
- ✓ Can move to cloud for better performance
- ✓ Simple to add new features

Economic Feasibility

Category	Cost
Developer's Salary	₹5 LPA for each (3 developers)
Development Laptops	₹60,000 each(3 laptops)
Cloud hosting (AWS)	₹90,000 (for 1 year)
Domain Name (e.g., .com)	₹1,500 (per year)
Internet	₹12,000
Third party services (APIs)	₹50,000
Miscellaneous(travelling, electricity ,etc)	₹20,000

Total Cost: ₹ 10,43,500

SDLC MODEL TO BE USED

- Software Development Life Cycle (SDLC) is a structured process for developing software efficiently and effectively.
- There are different types of SDLC models which provide varying approaches to managing and executing the software development process:
- Waterfall Model
- **Iterative Waterfall Model**
- Spiral Model
- V-Model(Verification & Validation)
- Agile Model
- Prototype Model
- RAD (Rapid Application Development)

Iterative Waterfall Model

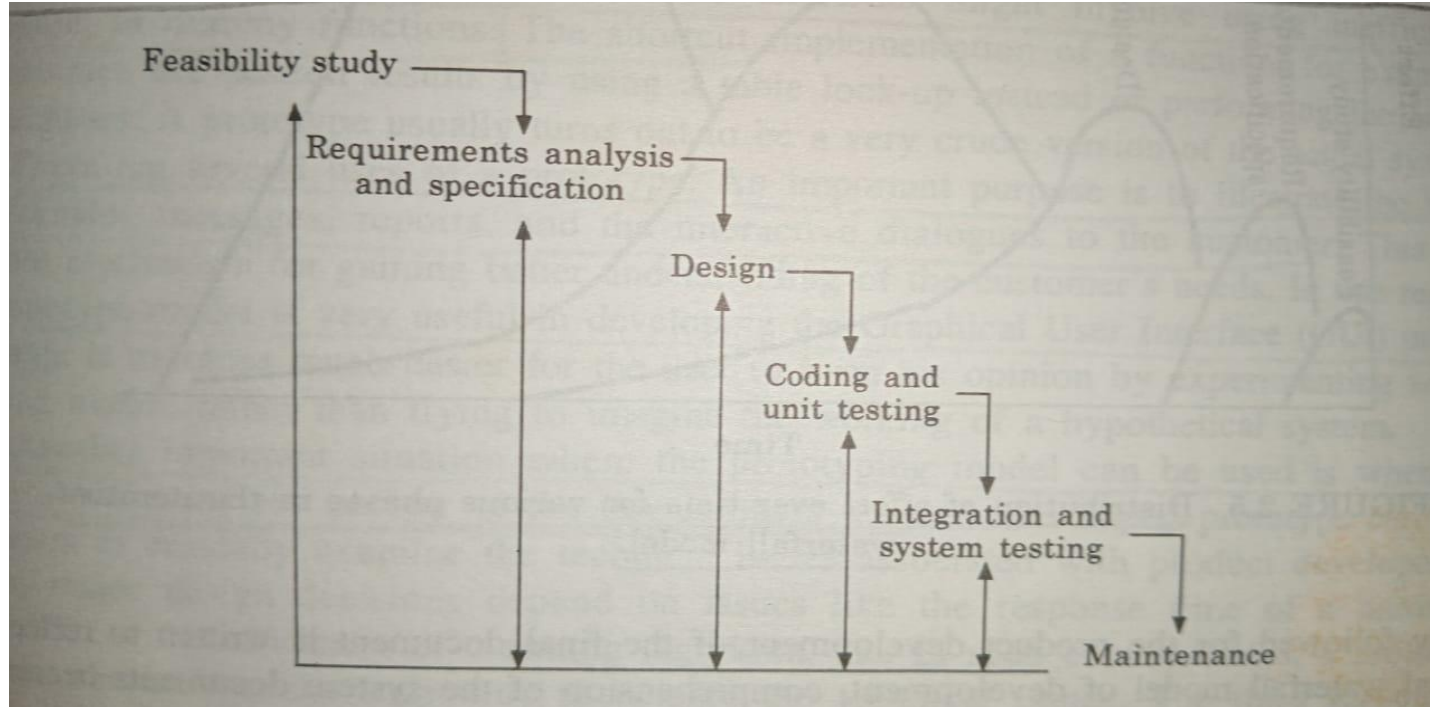


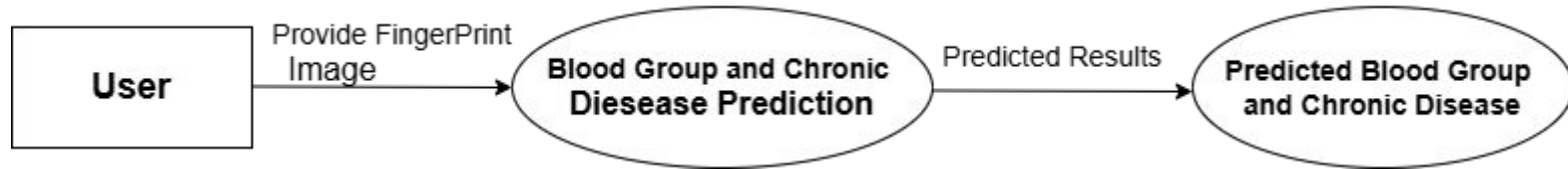
Image source: Rajib Mall's
Software Engineering book

Iterative Waterfall Model

- 1. Structured with Iterative Flexibility:** Combines a structured phase-by-phase approach with iterative feedback loops, making adjustments possible at certain stages without losing overall flow.
- 2. Clarity and Documentation:** Clear milestones and comprehensive documentation help maintain transparency in project tracking and development.
- 3. Simplified Management and Tracking:** Each phase flows in a linear sequence with designated checkpoints, making it easier to manage timelines and monitor progress.
- 4. Fixed Requirement Handling:** Ideal for projects with stable requirements, where extensive, ongoing changes are unnecessary.
- 5. Time-Effective Milestones:** Well-defined stages allow for effective time management, ensuring each part of the project is completed before moving on to the next.
- 6. Minimal Client Interaction:** Unlike Agile, which requires frequent client feedback, the iterative waterfall model allows development without continuous external input, reducing interruptions.

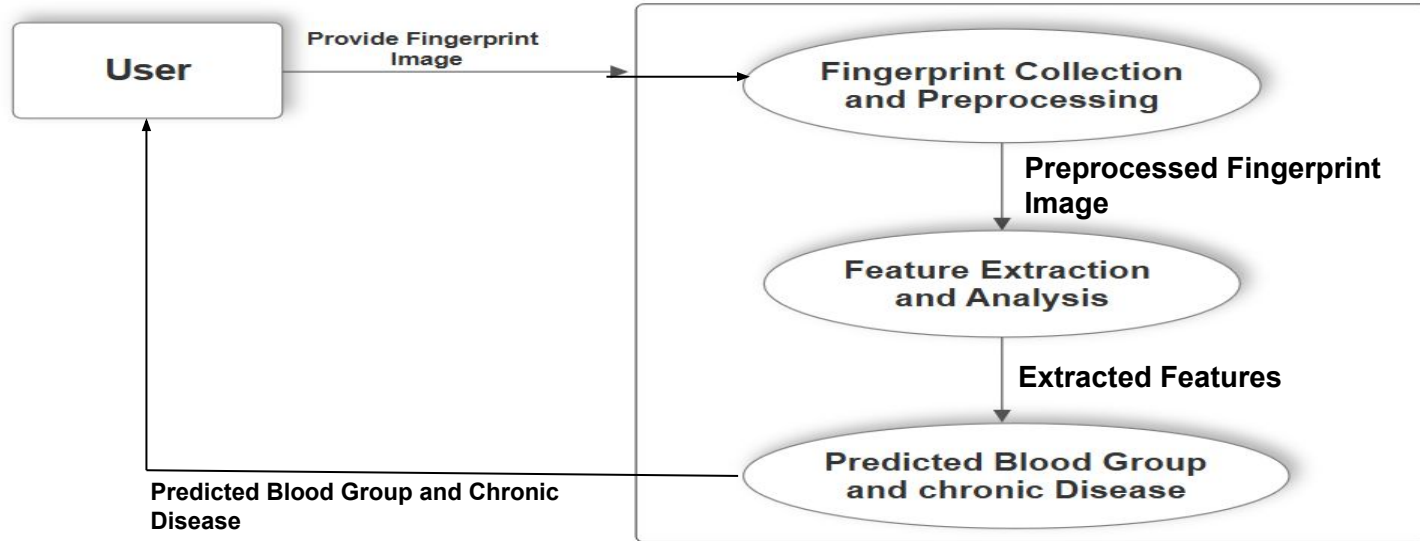
Data Flow Diagram

- Level 0 DFD – Fingerprint analysis and prediction System



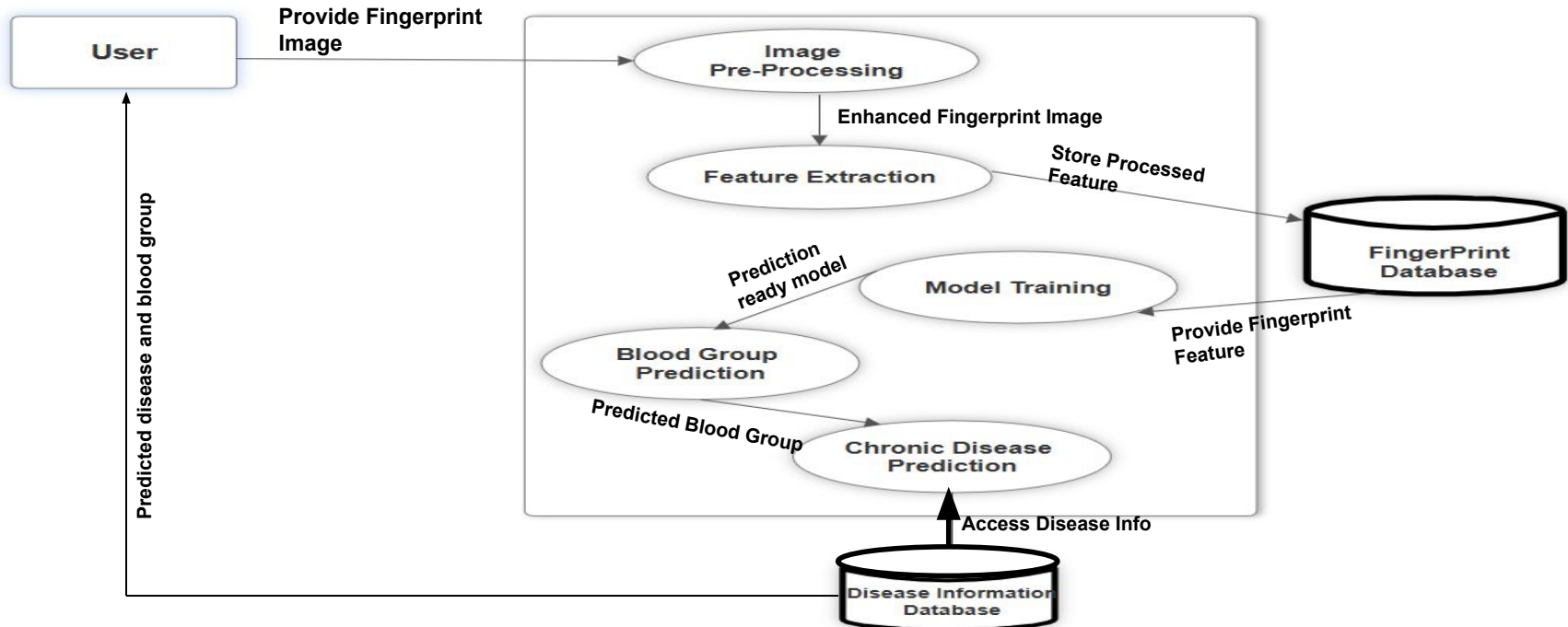
Data Flow Diagram

- Level 1 DFD – Fingerprint analysis and prediction System



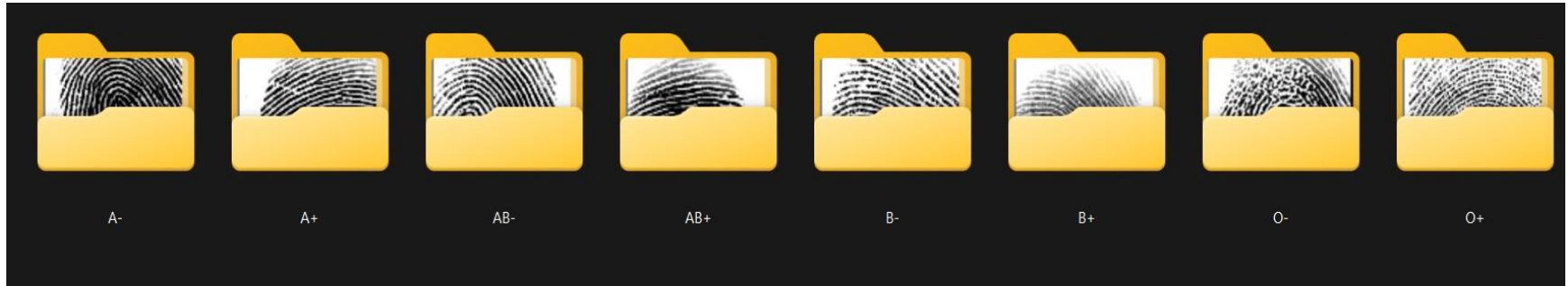
Data Flow Diagram

- Level 2 DFD – Fingerprint analysis and prediction System



Dataset

- we have a dataset organized by blood type, with separate folders for each blood group and Rh factor (A-, A+, AB-, AB+, B-, B+, O-, O+). Each folder contains fingerprint images associated with that particular blood type.
- Each folder has 1000 images(approximately)
- Dimension of image – 96 x 103 pixels
- Extension of image - .BMP (Bitmap images)
- Image type – Greyscale image
- Total dataset size – 6000 images [Dataset Folder](#)



Dataset Sample



Arch (A)



Loop (L)



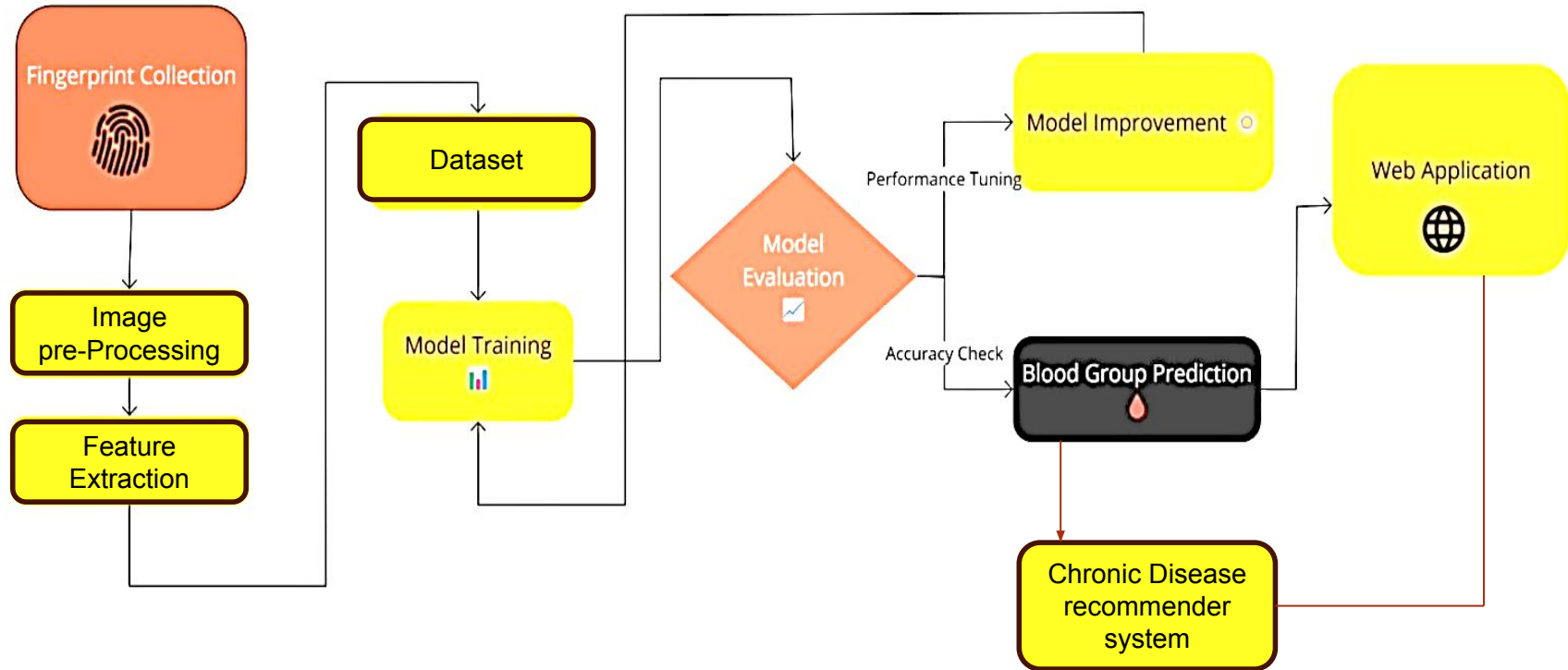
Whorl (W)



Dataset Features

Sr No	Fingerprint Attribute	Sr No	Fingerprint Attribute
1	CandId	15	FIRLength
2	FingerNo	16	DWTLevel1
3	FingerSize	17	DWTLevel2
4	MinValyThikness	18	DWTLevel3
5	MaxValyThikness	19	TotalLines
6	AverageVallyThikness	20	MinLineLength
7	MinRidgeThikness	21	MaxLineLength
8	MaxRidgeThikness	22	AvgLineLength
9	AverageRidgeThikness	23	FingerPattern
10	MinutieCount	24	DWTLevel4
11	MinMinutieAngle	25	DWTLevel5
12	MaxMinutieAngle	26	DWTLevel6
13	AvgMinutieAngle	27	RidgeBifurcationCount
14	FingerQuality	28	RidgeEndCount

Proposed System Architecture



Methodology

Data Collection

- Fingerprint images
- Blood Group labels (A,B,O,AB)

Data Processing

- Image Processing : Convert to grayscale , resize to standard size , and normalize pixel values
- Data Augmentation : Apply rotation , flipping , and scaling techniques to enhance model robustness

Feature Extraction: Extract key fingerprint features relevant to blood group prediction.

Dataset Creation: Organize images and extracted features into a structured dataset.

Methodology

Model building using ANN:

- Define a sequential neural network with an input layer, multiple hidden layers, and an output layer.
- Input layer: Accepts preprocessed fingerprint data (flattened image or extracted features).
- Output layer: Uses SoftMax activation for multi-class classification, giving probabilities for each blood group.
- Hidden layers: Consist of neurons with ReLU activation for non-linearity, allowing the model to learn complex patterns.

Model Training: Train a machine learning model on the dataset.

Model Evaluation : Assess model performance on a separate set using accuracy , precision, recall and F1 score metrices

Methodology

- **Blood Group Prediction:** The trained model predicts the blood group based on fingerprint features.
- **Web Application:** The entire system, including blood group prediction and disease recommendation, is deployed for user interaction.

Techniques used for Fingerprint Features extraction

- The technique is divided into two stages:

Stage 1: Preprocessing

- Enhances poor-quality fingerprint images.
- Employs image enhancement techniques to improve the clarity of ridge patterns.

Stage 2: Feature Extraction and Classification

1. Feature Extraction:

- Feature extraction involves identifying and isolating key details from the processed image.
- Types of features extracted will be minutiae points, ridge orientation, ridge frequency, core and delta points etc.
- We will use Crossing number algorithm to identify minutiae points.

2. Classification:

- It is the process of categorizing the fingerprint into its classification categories.
- Fingerprint classification categories: Arch, Loops, whorls, tented arches.

Algorithms used in Fingerprint Image Pre-Processing

- In order to start the feature extraction we first need to enhance the fingerprint image that is used for the feature extraction. This is done by pre-processing techniques on the image before it is used in further processes.
- The techniques are Binarization, filling of holes, Thinning of Image, dilation & then removal of unwanted portions. In case we want to find the minutiae points using valleys then we also need to do the inversion of the image along with the techniques provided above.



Fig. 1. Minutiae points in Fingerprint

1. Convert input image into Greyscale image:

- Converting an input fingerprint image to greyscale image reduces a coloured image (with three channels: Red, Green, Blue) to a single channel image where pixel values represent intensity levels (from black to white)
- It reduces the complexity of image, making it easier to process the tasks like edge detection, binarization, or feature extraction.
- Code Implementation using OpenCV:

```
original_image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
```

2. Binarization

- Binarization is the process where we change the input image to a binary image i.e. 0 or 1. We do so by first computing a threshold value and any pixel whose value is less than threshold is considered black while all the rest are considered white.
- There are two different algorithms to perform binarization: Manual Thresholding, Otsu algorithm.
- Manual Thresholding:

```
def binarize_image(image):  
    _, binary_image = cv2.threshold(image, thresh: 128, maxval: 255, cv2.THRESH_BINARY)  
    return binary_image
```

- Otsu algorithm:

```
def binarize_image(image, threshold=None):  
    if threshold is None:  
        threshold, _ = cv2.threshold(image, thresh: 0, maxval: 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)  
    _, binary_image = cv2.threshold(image, threshold, maxval: 255, cv2.THRESH_BINARY)  
    return binary_image
```


2. Binarization

- A comparison between the obtained results is presented in the below figure.

Original Image



Binary Image



Binary Image



Fingerprint image binarized with: A) Manual Thresholding B) Otsu Method

2. Binarization

- It should be pointed out that in this case binarization is an image segmentation method.
- It means that black pixels will represent fingerprint and white ones will correspond to the background.
- The best results were obtained with Otsu method.
- By this algorithm all elements of the fingerprint are clearly visible.
- Selected method allows to avoid inaccurate representation of a fingerprint
- However, a few elements that are not a part of a fingerprint have black colour.

3 - Filing of Holes

- Here, we fill all the holes that are present in the ridges of the fingerprint.
- We do so because in case if we don't fill these holes , each of these holes can also be looked as two bifurcations near to each other. So when we try to compute the minutiae points in the fingerprint then a lot of wrong entries will be encountered .
- This will decrease the accuracy of minutiae points that are extracted from the fingerprint.
- Code Implementation:

```
def fill_holes(binary_image):  
    inverted_image = cv2.bitwise_not(binary_image)  
    filled_image = cv2.morphologyEx(inverted_image, cv2.MORPH_CLOSE, np.ones(shape: (3, 3), np.uint8))  
    filled_image = cv2.bitwise_not(filled_image)  
    return filled_image
```

3 - Filing of Holes

- Resultant output:

Filled Image



4. Thinning

- Thinning is the image where we transform the image in such a way that the width of the ridges is changed to one pixel.
- This is done because it is easy to find minutiae points in the thinned image as compared to the original image, as we can easily check the pattern of the pixel in the thinned image because width after thinning is one pixel.
- Code Implementation:

```
def thinning(filled_image):  
    thinned_image = cv2.ximgproc.thinning(filled_image, thinningType=cv2.ximgproc.THINNING_ZHANGSUEN)  
    return thinned_image
```

4. Thinning

- Resultant Output :

Thinned Image



5. Dilation

- It is a morphological operation that expands white regions in binary image
- It connects broken ridges or highlight significant features
- It adds pixels to the boundary of images
- This expands ridge structures and can be particularly useful for enhancing ridges

Code:

```
import cv2
import numpy as np

# Load the image in grayscale
image = cv2.imread("C:/Users/palak/Downloads/Fingerprint img.bmp", cv2.IMREAD_GRAYSCALE)

# Apply dilation with a 5x5 kernel
kernel = np.ones((5, 5), np.uint8)
dilated_image = cv2.dilate(image, kernel)
# Display the original and dilated images
cv2.putText(image, 'Original Image', (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (255), 2, cv2.LINE_AA)
cv2.putText(dilated_image, 'Dilated Image', (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (255), 2, cv2.LINE_AA)
cv2.imshow("Dilation", np.hstack((image, dilated_image)))
cv2.waitKey(0)
cv2.destroyAllWindows()
```


Output:



6. Thinning again on Dilated fingerprint

- Thinning reduces the thickness of objects (like ridges) while preserving their core structure.
- It erodes pixels and removes excess thickness, often using morphological operations like erosion and dilation or skeletonization.
- After dilation, thinning removes unwanted extra pixels, leaving behind the essential features like the fingerprint ridges in a thinner form.

Code:

```
> import cv2
import numpy as np

# Load the image in grayscale
image = cv2.imread("C:/Users/palak/Downloads/Fingerprint img.bmp", cv2.IMREAD_GRAYSCALE)
# Apply dilation
kernel = np.ones((5, 5), np.uint8)
dilated_image = cv2.dilate(image, kernel)
# Perform thinning using morphological operations
thinned_image = cv2.morphologyEx(dilated_image, cv2.MORPH_ERODE, np.ones((3, 3), np.uint8))

# Display dilated and thinned images
cv2.putText(dilated_image, 'Dilated Image', (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (255), 2, cv2.LINE_AA)
cv2.putText(thinned_image, 'Thinned Image', (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (255), 2, cv2.LINE_AA)
cv2.imshow("Thinning", np.hstack((dilated_image, thinned_image)))
cv2.waitKey(0)
cv2.destroyAllWindows()
```

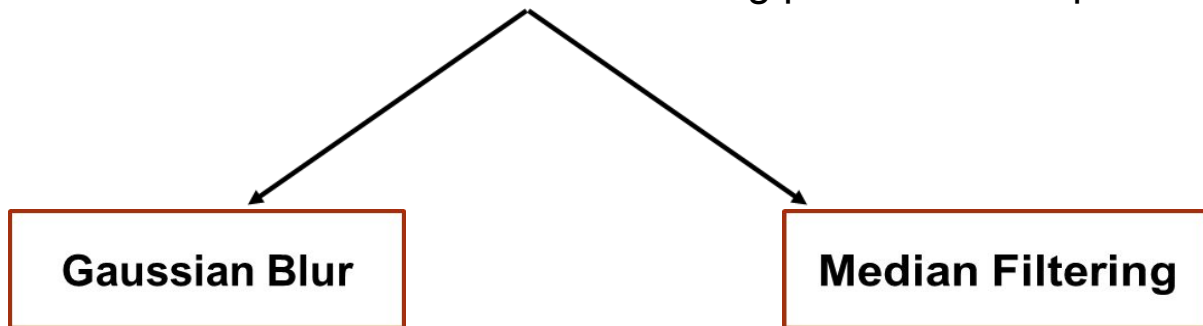
Output:

Thinning



7. Removing unwanted patterns

- Firstly, we have to remove the unwanted patterns i.e., Noise from the images which occurred due to smudges, uneven lighting background artifacts, etc
- The process of Noise Removal involves filtering process/techniques:



Gaussian Blur

- It is a low pass filter that smooths out the image by averaging pixel values with their neighbors.
- It works by convolving the image with Gaussian Kernel which gives more weight to pixels near the center & reduce weight of pixels further away.

Code:

```
import cv2

# Load the image in grayscale
image = cv2.imread("C:/Users/palak/Downloads/Fingerprint img.bmp", cv2.IMREAD_GRAYSCALE)

# Apply Gaussian Blur with a larger kernel size and standard deviation
# Larger kernel (e.g., 11x11) increases the blur effect
blurred_image = cv2.GaussianBlur(image, (15, 15), 0)

# Add headings on the images
cv2.putText(image, 'Original Image', (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (255), 2, cv2.LINE_AA)
cv2.putText(blurred_image, 'Gaussian Blurred Image', (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (255), 2, cv2.LINE_AA)

# Display both images side by side
cv2.imshow("Gaussian Blur", cv2.hconcat([image, blurred_image]))
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output :

Gaussian Blur



Median Filtering

- This filter removes salt & pepper noise.
- It works by replacing each pixel value with median of its neighboring pixels which helps to preserve edges while removing sharp isolated noise points.

Code:

```
import cv2

# Load the image in grayscale
image = cv2.imread("C:/Users/palak/Downloads/Fingerprint img.bmp", cv2.IMREAD_GRAYSCALE)
# Apply Median Filtering with a larger kernel size
# Larger kernel size (e.g., 9) increases the smoothing effect
median_filtered_image = cv2.medianBlur(image, 9)

# Add headings on the images
cv2.putText(image, 'Original Image', (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (255), 2, cv2.LINE_AA)
cv2.putText(median_filtered_image, 'Median Filtered Image', (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (255), 2, cv2.LINE_AA)

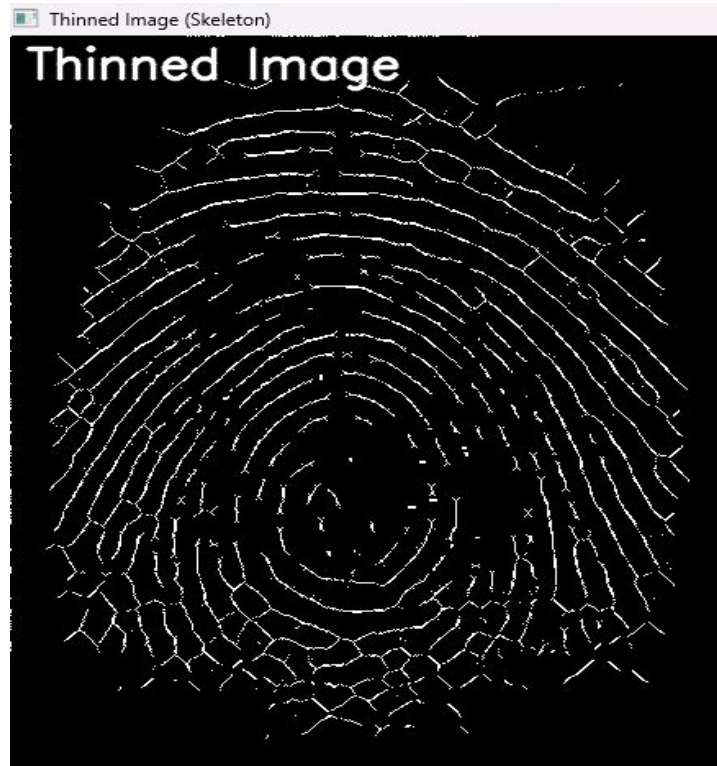
# Display both images side by side
cv2.imshow("Median Filtering", cv2.hconcat([image, median_filtered_image]))
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output :

Median Filtering



Final Image (Thinned image after applying filters) for Minutiae point detection and further processing for classification :



8. Minutiae point Detection

After finishing all the preprocessing process, we get the final image from which the minutiae points can be extracted. Thus we can easily find the minutiae points by using the **Crossing Number Algorithm**.

In this we take a 3*3 map and move it across the whole image. In this the properties of the point P is calculated by its crossing no i.e. the no of points from P1 to P8 that are black.

P1	P2	P3
P8	P	P4
P7	P6	P5

P: Central pixel (ridge point being tested).

P1 to P8: 8-neighboring pixels in clockwise order.

The **crossing number** is calculated as:

$$CN = \frac{1}{2} \sum_{i=1}^8 |P_i - P_{i+1}|$$

(P9 = P1 to close the loop)

- **P1, P2, ..., P8** represent the binary pixel values (0 = background, 1 = ridge pixel).
- The absolute difference $|P_i - P_{i+1}|$ captures transitions between **0 and 1**.
- P9 is set equal to P1 to form a circular structure.

Steps to Perform CN Algorithm

1. **Input:** A binary image where ridges are white (1) and the background is black (0).
2. For every ridge pixel PPP (white pixel = 1), examine its **8-neighborhood** (P1 to P8).

3. Calculate the **Crossing Number (CN)** using the formula above.

4. **Classify the Minutiae Point:**

- **CN = 1** → Ridge Ending
- **CN = 3** → Ridge Bifurcation
- **CN = 4** → Crossing or Loop
- **CN = 0** or other values → Not a minutiae point

Example Calculation

Let's assume the values in the 3x3 neighborhood are:

P1 = 0, P2 = 1, P3 = 0

P8 = 0, P = 1, P4 = 0

P7 = 0, P6 = 1, P5 = 0

$$\begin{array}{|l} \overrightarrow{\quad} \\ P_1 = 0, P_2 = 1, P_3 = 0 \\ P_8 = 0, P_7 = 1, P_4 = 0 \\ P_7 = 0, P_6 = 1, P_5 = 0 \\ \overleftarrow{\quad} \end{array}$$

The values in circular order = $P_1, P_2, P_3, P_4, P_5,$
 $P_6, P_7, P_8, P_1.$

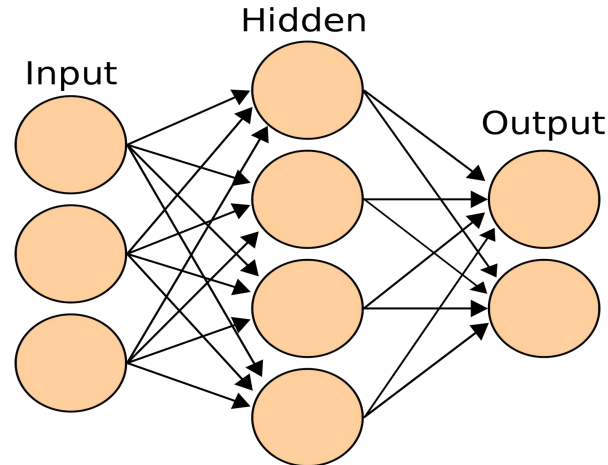
Substituting values in formula :-

$$\begin{aligned} CN &= \frac{1}{2} (|P_1 - P_2| + |P_2 - P_3| + |P_3 - P_4| + |P_4 - P_5| + \\ &\quad |P_5 - P_6| + |P_6 - P_7| + |P_7 - P_8| + |P_8 - P_1|) \\ &= \frac{1}{2} \times (1 + 1 + 0 + 0 + 1 + 1 + 0 + 0) \\ &= \frac{1}{2} \times 4 = 2. \end{aligned}$$

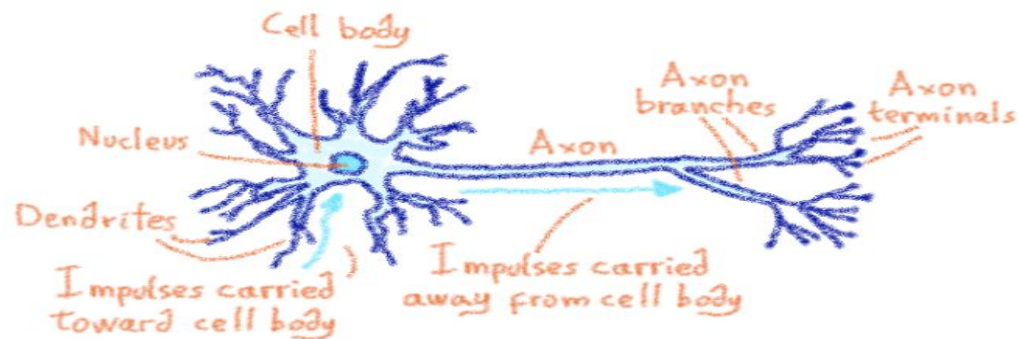
So, Here $CN = 2$ (Not a minutiae point)

NEURAL NETWORK

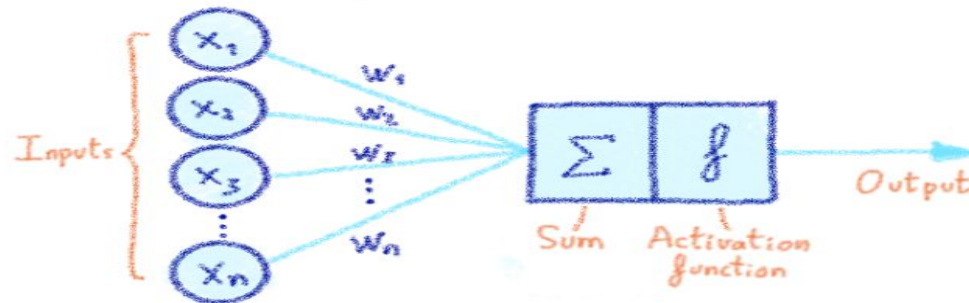
- Neural Network is an architecture which is inspired by human brain
- The human brain can be described by biological neural network.
- It is a set of algorithms designed to recognize patterns, model complex relationships, and solve various problems, especially in areas such as machine learning, deep learning, and artificial intelligence.



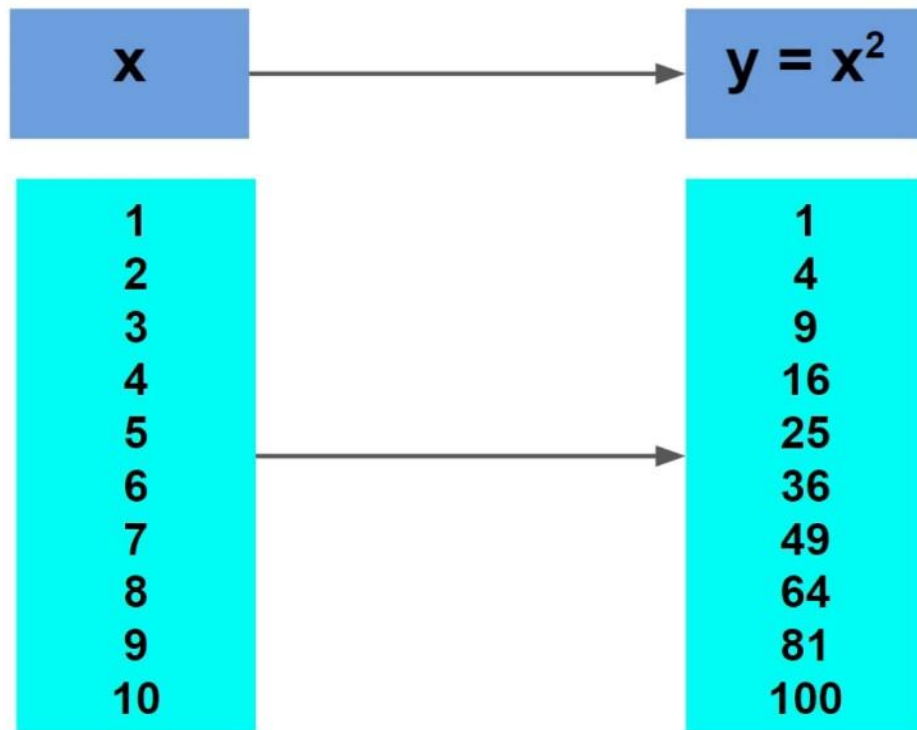
Biological Neuron



Artificial Neuron



Let's make a neural network



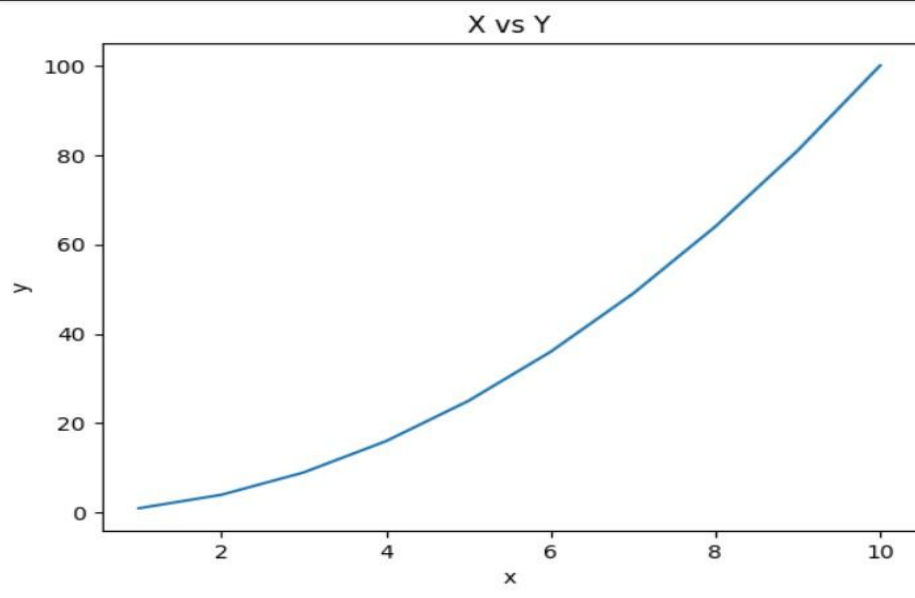
Let's make a neural network

```
import tensorflow as tf
import numpy as np
import pylab
from tensorflow import keras
```

Generated code may be subject to a license | SkillsSquad/Deep-Learning

```
x = np.array([1,2,3,4,5,6,7,8,9,10])
y = np.array([1,4,9,16,25,36,49,64,81,100])
pylab.plot(x,y)
pylab.xlabel('x')
pylab.ylabel('y')
pylab.title('X vs Y')
```


Let's make a neural network



Generated code may be subject to a license | Barbarpotato/DeepLearning.AI-TensorFlow-Developer-Professional-Certificate

```
# train the neural network
# no of hidden layers = 1
# optimiser = stochastic gradient descent
# epoches = 1000
```

```
model = tf.keras.Sequential([keras.layers.Dense(units=1, input_shape=[1])])
model.compile(optimizer='sgd', loss='mean_squared_error')
model.fit(x, y, epochs=1000)
```

Let's make a neural network

```
Epoch 996/1000
1/1 ————— 0s 45ms/step - loss: 52.8266
Epoch 997/1000
1/1 ————— 0s 61ms/step - loss: 52.8264
Epoch 998/1000
1/1 ————— 0s 54ms/step - loss: 52.8262
Epoch 999/1000
1/1 ————— 0s 47ms/step - loss: 52.8259
Epoch 1000/1000
1/1 ————— 0s 142ms/step - loss: 52.8257
<keras.src.callbacks.history.History at 0x78de548f35b0>
```

```
# predict
```

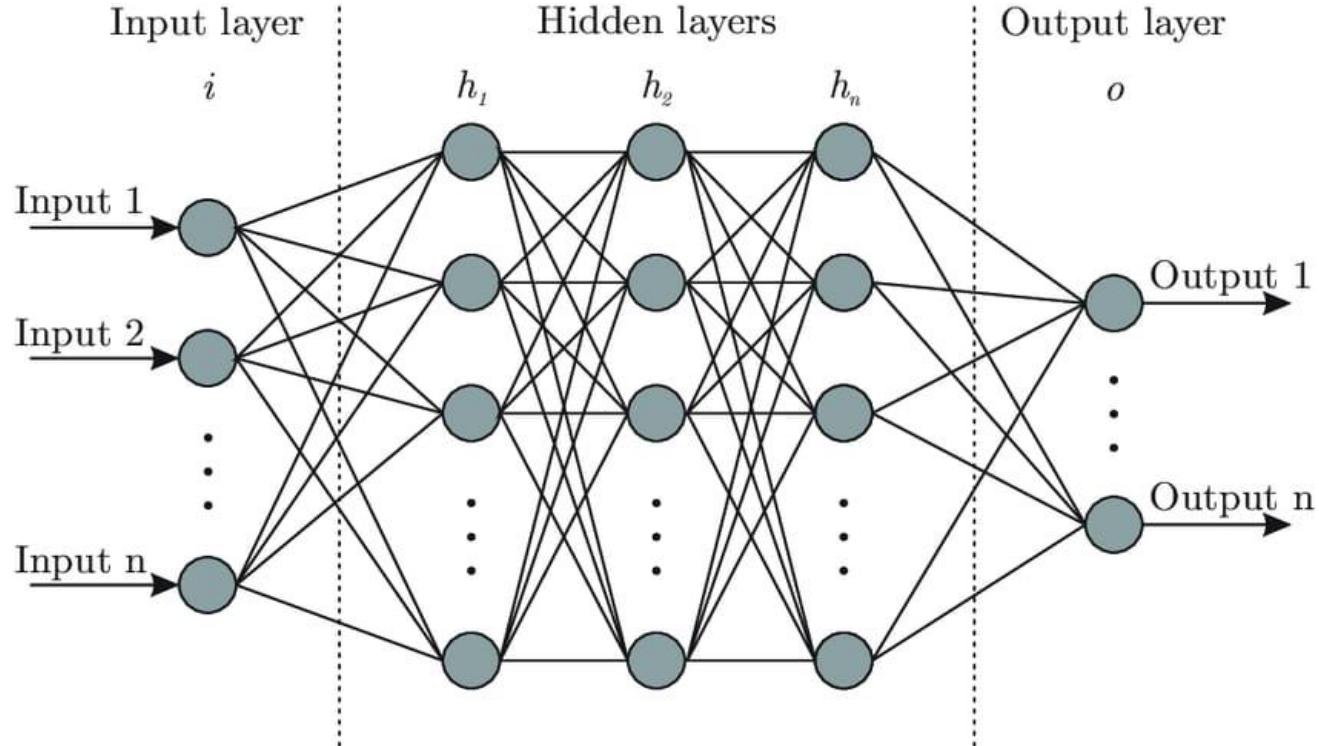
```
print(model.predict(np.array([8])))
```

```
1/1 ————— 0s 20ms/step
[[65.94853]]
```

Artificial Neural Network

- An ANN is defined as an information processing model that is inspired by the way biological nervous systems, such as the brain processes the information.
- ANN is inspired by biological neural network which contains **Artificial Neurons**
- An ANN is composed of large number of highly interconnected processing units (neurons) working in unison to solve the specific problems.
- Like human being, ANN learns by example.
- An ANN is configured for a specific application such as spam classification , Face Recognition, pattern recognition through a learning process.
- Each neuron is connected with the other by a connection link.
- Each connection link is associated with a weights which contains information about the input signals.
- This information is used by the neuron Network to solve a particular problem

Architecture of ANN



Model of Artificial Neural Network

The Following diagram represents the general model of ANN followed by its processing:

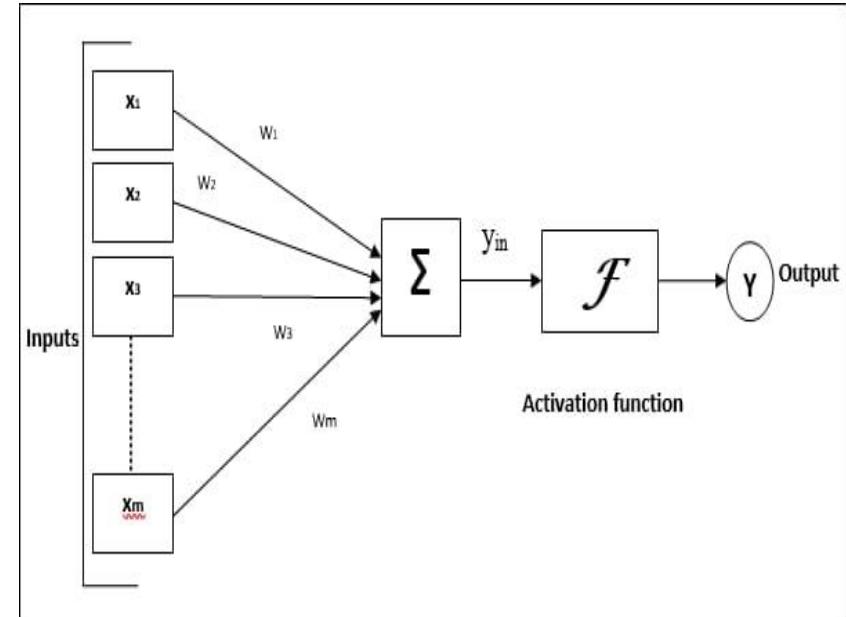
For the general model of ANN, the net input can calculate as follows:

$$y_{in} = x_1.w_1 + x_2.w_2 + x_3.w_3 + \dots + x_m.w_m$$

i.e. Net Input $y_{in} = \sum_i^m x_i.w_i$

The output can be calculated by applying the activation function over the net input.

$$Y = F(y_{in})$$



Architecture of ANN

Artificial Neural network primarily consists of three layers:

1. **Input Layer:** It accepts input in several different formats provided by the programmer
2. **Hidden Layer:** It is present in between input and output layers. It performs all the calculations to find hidden features and patterns.
3. **Output Layer:** The input goes through a series of transformations using the hidden layer, which finally results in output that is conveyed using this layer.

Numerical on ANN (Hamming Distance)

Q. Consider a 3 input neuron. The output is 1 when the input (X1, X2, X3) is 111 or 101. and output is 0 when the input (X1, X2, X3) is 000 or 001.

Find the output for 010 using prediction method.

$3 \rightarrow 2^3 = 8$

000	→	0	✓
001	→	0	
<u>010</u>	→	0	
011			
100			
101	→	1	
110			
111	→	1	

010

- 000 → 1 ✓
- 001 → 2
- 101 → 3
- 111 → 2

Basic Components or Entities of ANN

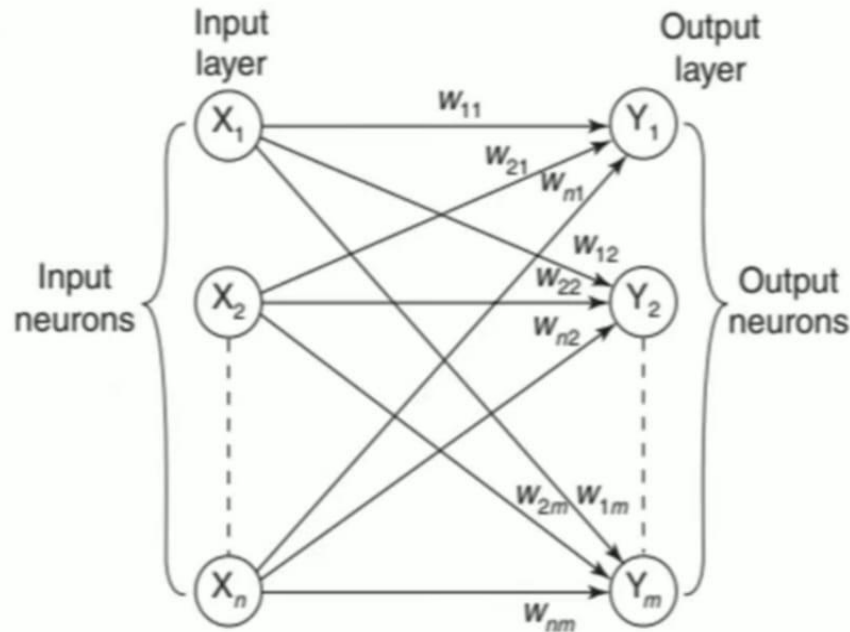
The models of ANN are specified by three basic entities namely:

1. The model's synaptic interconnections.
2. The training or learning rules adopted for updating and adjusting the connection weights.
3. Their activation Functions.

1. Connections

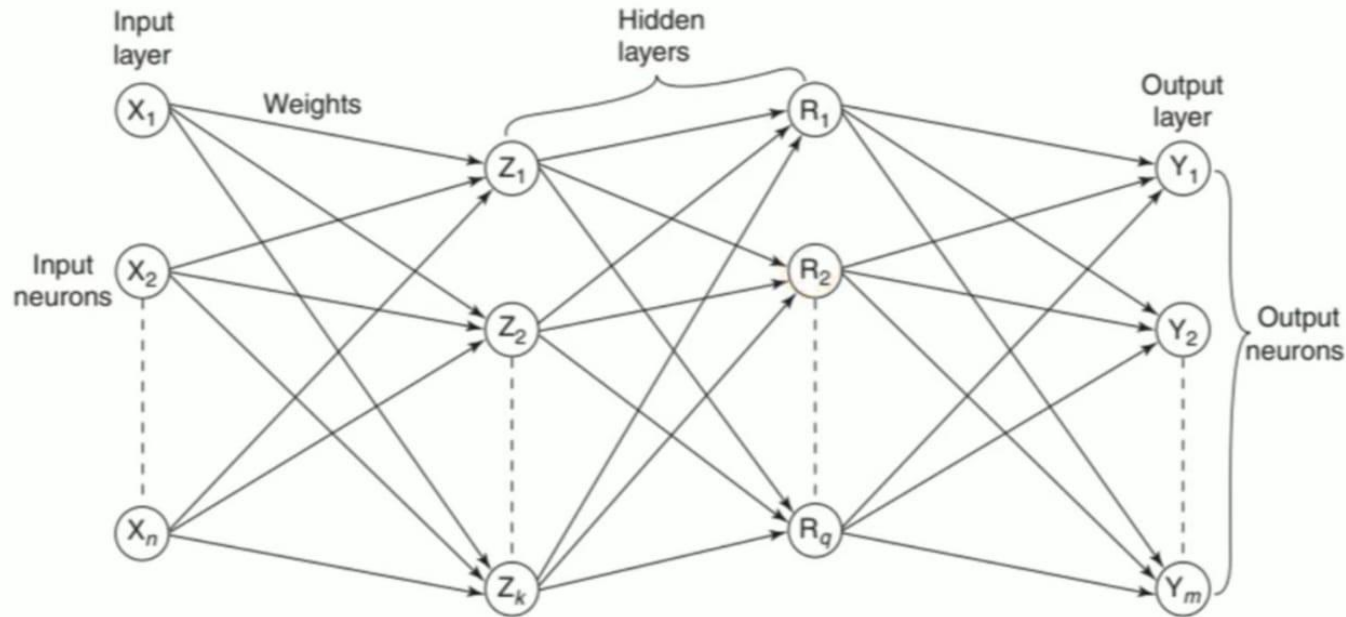
- There exists five basic types of neuron connection architectures. They are:
 1. Single - Layer feed - forward network
 2. Multilayer feed-forward network
 3. Single node with its own feedback
 4. Single layer recurrent network
 5. Multilayer recurrent network

1. Single Layer Feed-Forward Network



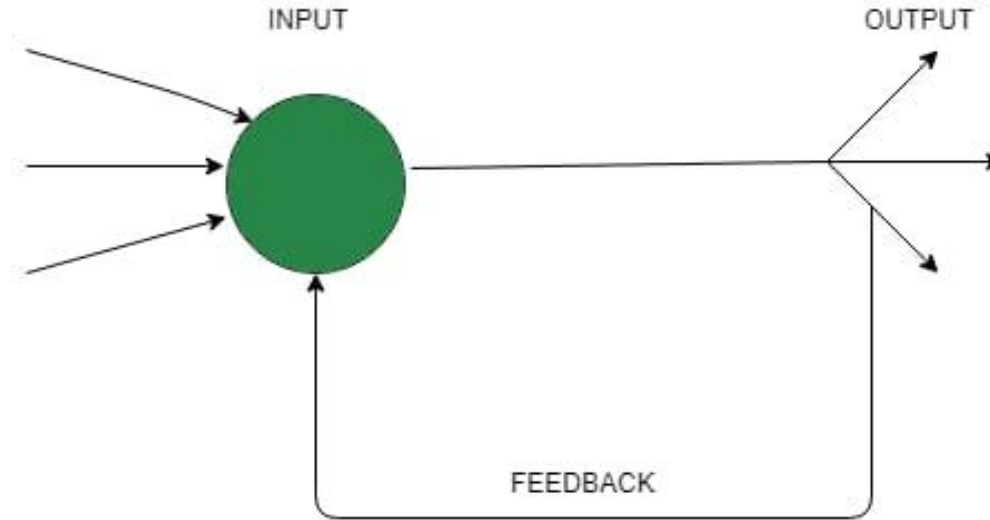
Single-layer feed-forward network.

2. Multilayer Feed-Forward neural Network

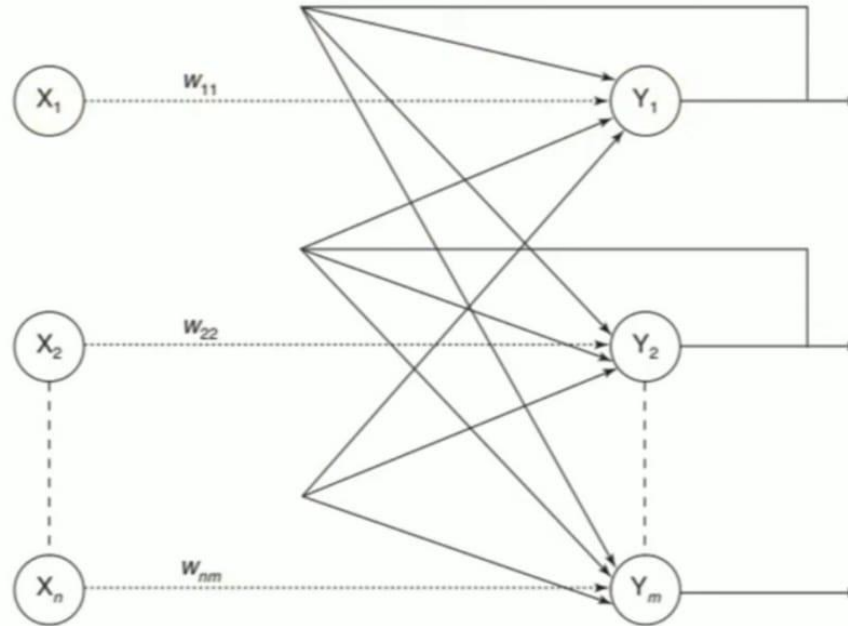


Multilayer feed-forward network.

3. Single node with its own feedback

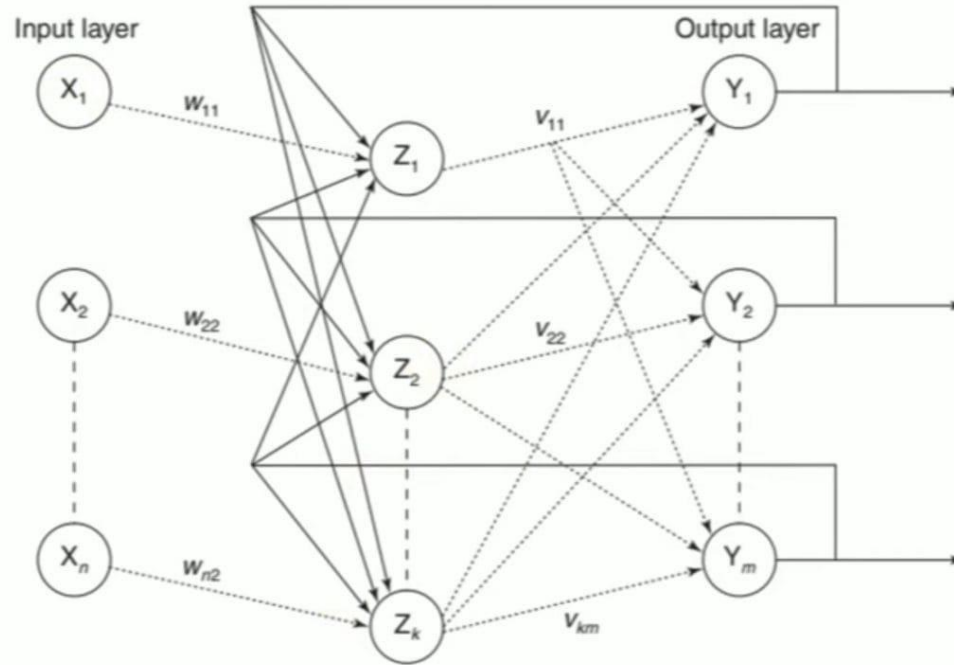


4. Single layer Recurrent network



Single-layer recurrent network.

5. Multilayer recurrent network

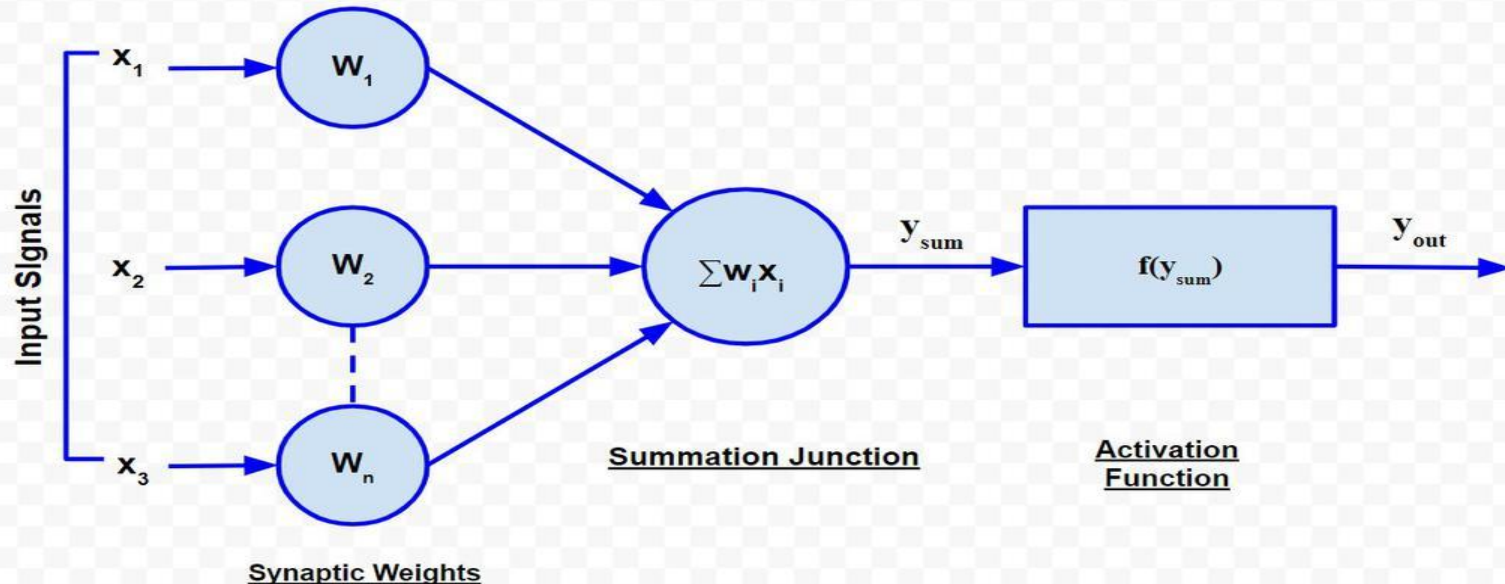


2. Learning of ANN

- The property of ANN is its capability to learn.
- Learning of training is a process by means of which a neural network adapts itself to a stimulus by making proper parameter adjustments, resulting in the production of desired response.
- Broadly, there are two kinds of learning of ANNs:
 1. **Parameter learning:** It updates the connecting weights in a neural network
 2. **Structure learning:** It focuses on the change in network structure (which include the number of processing elements as well as their connection types).

3. Activation Functions

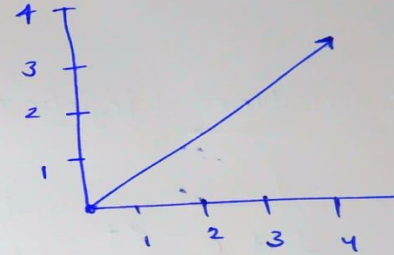
- In Neural Networks the Activation function is usually an abstraction representation the rate of action potentially firing in the cell
- The function is binary. That means either the neuron is firing or not



Types of Activation Functions

Threshold Activation function

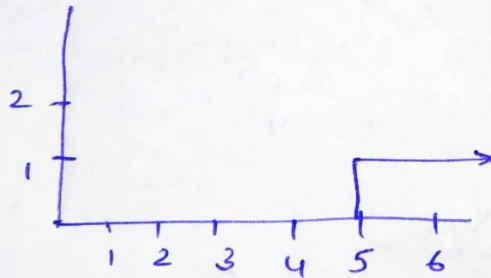
$$\phi(t) = \begin{cases} t & \text{if } t \geq 0 \\ 0 & \text{if } t < 0 \end{cases}$$



Another type

$$\phi(x) = \begin{cases} 1 & \text{if } x \geq \theta \\ 0 & \text{if } x < \theta \end{cases}$$

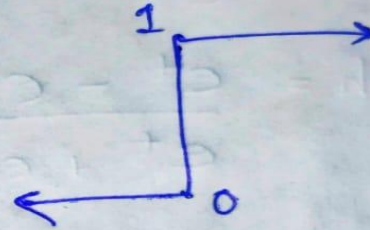
θ = Given value (5)



Types of Activation Functions

Step function

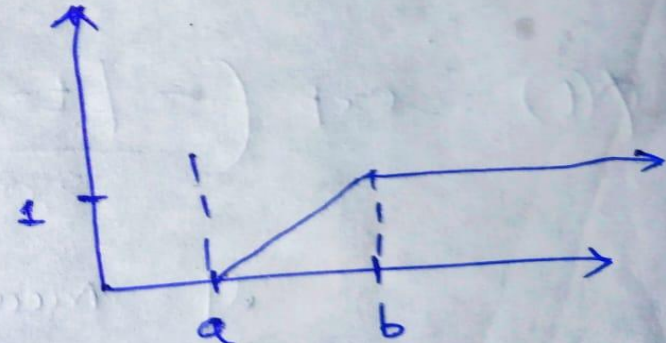
$$\phi(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$$



Piece wise linear function

$$\phi(t) = \begin{cases} 0 & \text{if } t \leq a \\ t & \text{if } a < t < b \\ 1 & \text{if } t \geq b \end{cases}$$

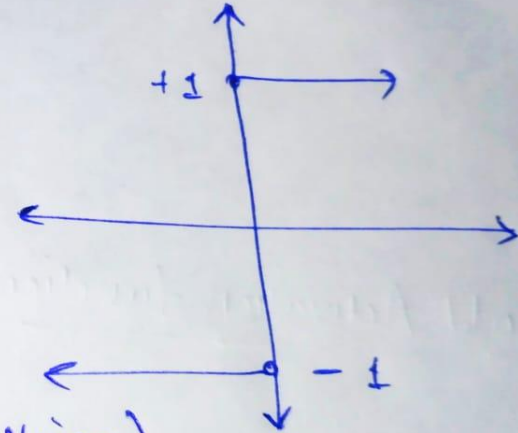
(a & b are given)



Types of Activation Functions

Signum function

$$\phi(x) = \begin{cases} +1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$



Sigmoidal function (Logistic or Squashing)

$$\phi(t) = \frac{1}{1 + e^{-\alpha t}} \quad \text{where } \alpha = \text{slope parameter}$$

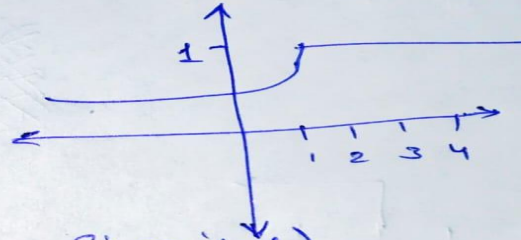
Types of Activation Functions

Sigmoidal function (Logistic or Squashing)

$$\phi(t) = \frac{1}{1 + e^{-\alpha t}} \quad \text{where } \alpha = \text{slope parameter}$$

① unipolar sigmoidal (log-Sigmoidal)

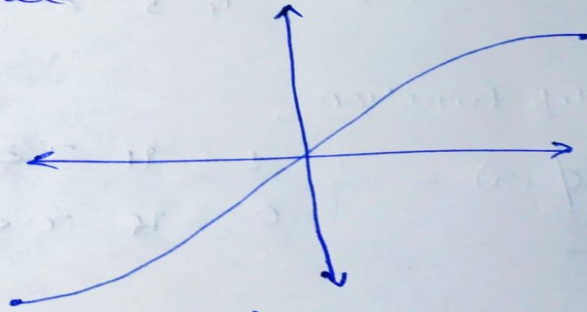
$$\phi(t) = \frac{1}{1 + e^{-\alpha t}}$$



② Bipolar Sigmoidal (Tan-Sigmoidal)

$$\phi(t) = \tanh(t)$$

$$\tanh = \frac{e^t - e^{-t}}{e^t + e^{-t}}$$



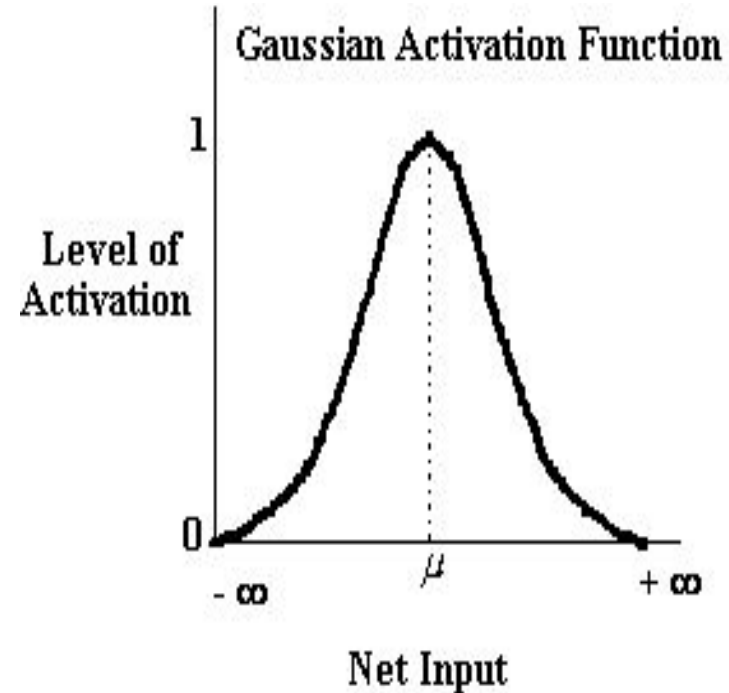
Types of Activation Functions

Gaussian function (Radial Basis function)

$$\phi(t) = \exp\left(-\frac{|t-\mu|}{2\sigma^2}\right) = e^{\left(\frac{-|t-\mu|}{2\sigma^2}\right)}$$

μ = Mean

σ = Standard deviation



Firing Rules

Neuron has 2 mode of operation

- 1 Training Mode
 - 2 Using / Prediction mode
-
- Training mode – in training mode the neuron can be trained to fire or not fire for a particular input pattern
 - Prediction mode – in this mode , it identify the input pattern and try to output the associated output pattern
 - Firing rule determines weather a neuron should fire or not for any input pattern

STUDY OF ALGORITHMS OF ML:

- There are several **algorithms** of **machine learning** which we can use in our project.

Name of such **ML algorithms** are as follows

- **CNN** (Convolutional Neural Network)
- **ANN** (Artificial Neural Network)
- **KNN** (K-nearest Neighbours)
- **SVM**(Support Vector machine)
- **Decision Trees**

ANN and **CNN** would be the most relevant for our project .

Lets study in detail about CNN & ANN & decide which will follow all the factors required and will be suitable 

CNN (Convolutional Neural Network)

The Convolutional layer

The Pooling layer

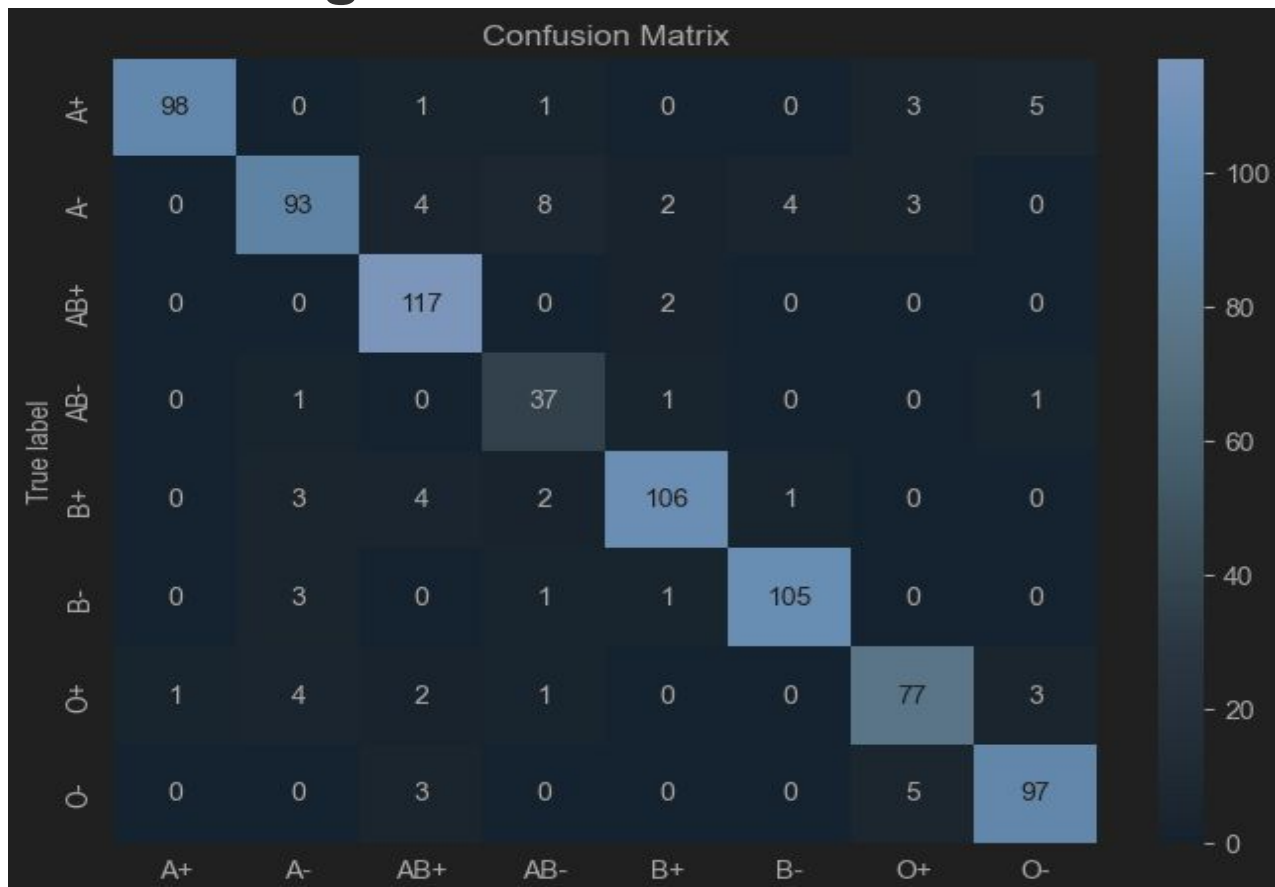
The output / fully connected layer

- **Introduction**
 - CNNs are Deep learning models
- **Purpose**
 - Image recognition and processing
- **Structure**
 - consist of layers such as convolutional layers, pooling layers, and fully connected layers.
- **Convolution Operation**
 - apply filters (kernels) to input images to extract features such as edges, textures, and patterns.
- **Pooling**
 - This step reduces the spatial dimensions (downsampling) of the image, making the model more efficient.
- **Applications**
 - Used in fields like computer vision, autonomous vehicles, and medical image analysis.

Reasons of using ANN instead of CNN

- The project involves manually classifying data, making ANN more suitable than CNN.
- ANN allows greater control over feature selection, aligning with the project's requirements.
- CNN's automatic feature extraction is unnecessary as features are pre-processed manually.
- ANN provides a straightforward approach, avoiding CNN's complex convolution and pooling layers.
- The project's primary focus is on non-image data classification, where ANN excels.

Reasons of using ANN instead of CNN



Language to be used

Python: Python is a most popular, easy to learn language with extensive libraries for machine learning, image processing.

Libraries to be used:

Image Processing and Preprocessing

1. OpenCV

- For image preprocessing (resizing, filtering, edge detection).
- Example: Enhancing fingerprint features.

2. Pillow (PIL)

- For image loading, manipulation, and conversion.
- Example: Converting images to grayscale.
- Installation: `pip install pillow`

Machine Learning and ANN

1. TensorFlow

- For building, training, and deploying neural networks.
- Example: Designing your ANN for blood group detection.

2. PyTorch

- Another framework for building deep learning models.
- Example: Alternative to TensorFlow with dynamic computational graphs.

Language to be used

3.Keras

- High-level API for TensorFlow, simplifies neural network creation.
- Example: Quickly building and training ANN models.

4.scikit-learn

- For preprocessing, feature selection, and simple machine learning models.
- Example: Scaling data, evaluating models, and testing classifiers.

Data Visualization and Analysis

1.Matplotlib

- For visualizing data and model performance (e.g., loss and accuracy plots).
- Example: Plotting fingerprint features or training graphs.

2.Seaborn

- For advanced data visualizations.
- Example: Correlation heatmaps for features.

Hardware Software Requirements

Hardware Requirements

- System : Pentium i3 Processor
- HDD : 500 GB
- RAM : 4 GB

Software Requirements

- OS : Windows , Linux
- Language : Python 3.10.9 , HTML , CSS , JS
- Framework : Flask

Objective , Scope of project

- Development of an Automated System

- The project will focus on designing and implementing an automated system capable of detecting blood groups (ABO and Rh) from images of blood samples using image processing techniques and convolutional neural networks (CNNs).

- Application In Healthcare

- The system is intended for use in hospitals, blood banks, and other healthcare settings where rapid and accurate blood typing is essential. The project could explore potential deployment in emergency situations, rural areas, or mobile medical units where access to laboratory testing is limited.

- Future Expansion

- The project has potential for future expansion to include other blood-related diagnostics, further optimization of image analysis algorithms, and the integration of additional blood parameters beyond ABO and Rh typing.

Objective , Scope of project

- Integration with cloud based services to store and share data
 - The project can explore integration with cloud-based platforms for storing blood group data securely.
- Scalability for mass blood testing
 - The system could be designed to scale for large-scale blood typing campaigns, such as blood donation drives or mass health check-ups, where rapid processing of large numbers of blood samples is required.
- Realtime emergency blood group determination
 - In emergency medical scenarios like accidents, surgeries, or urgent transfusions, the system can provide rapid and real-time blood group detection, helping healthcare professionals make immediate decisions.
- Reduction of Human error in blood typing
 - By automating blood group detection, the system aims to reduce human errors that can occur during manual blood typing. This will increase the reliability of test results, especially in high-pressure situations where human fatigue or oversight might occur.

Graphical User Interface

✓ Developed a Graphical User Interface (GUI)

- Create a **user-friendly** interface using **Flask**.
- Provide **smooth navigation** between different sections (Home, Login, Sign-Up).

✓ Implemented Fingerprint-Based Blood Group Detection

- Use **image processing techniques** to analyze fingerprint patterns.
- Extract features from **ridges and valleys** of fingerprints for blood group classification.

✓ Integrated a Backend System for User Authentication

- Implement **user registration and login** with a database.
- Ensure that users' data is securely **stored in SQLite** using Flask-SQLAlchemy.

✓ Store and Manage User Data Efficiently

- Design a **relational database schema** to store **user details** (name, email, password).
- Use **Flask-SQLAlchemy ORM** to interact with the database.

Technologies Used

- ◆ **Frontend (Graphical User Interface - GUI)**

- ✓ **Pillow (PIL), OpenCV** → Handles images (background images, fingerprint display).

- ◆ **Backend (API & Database Management)**

- ✓ **Flask** → Python-based micro web framework to handle HTTP requests.

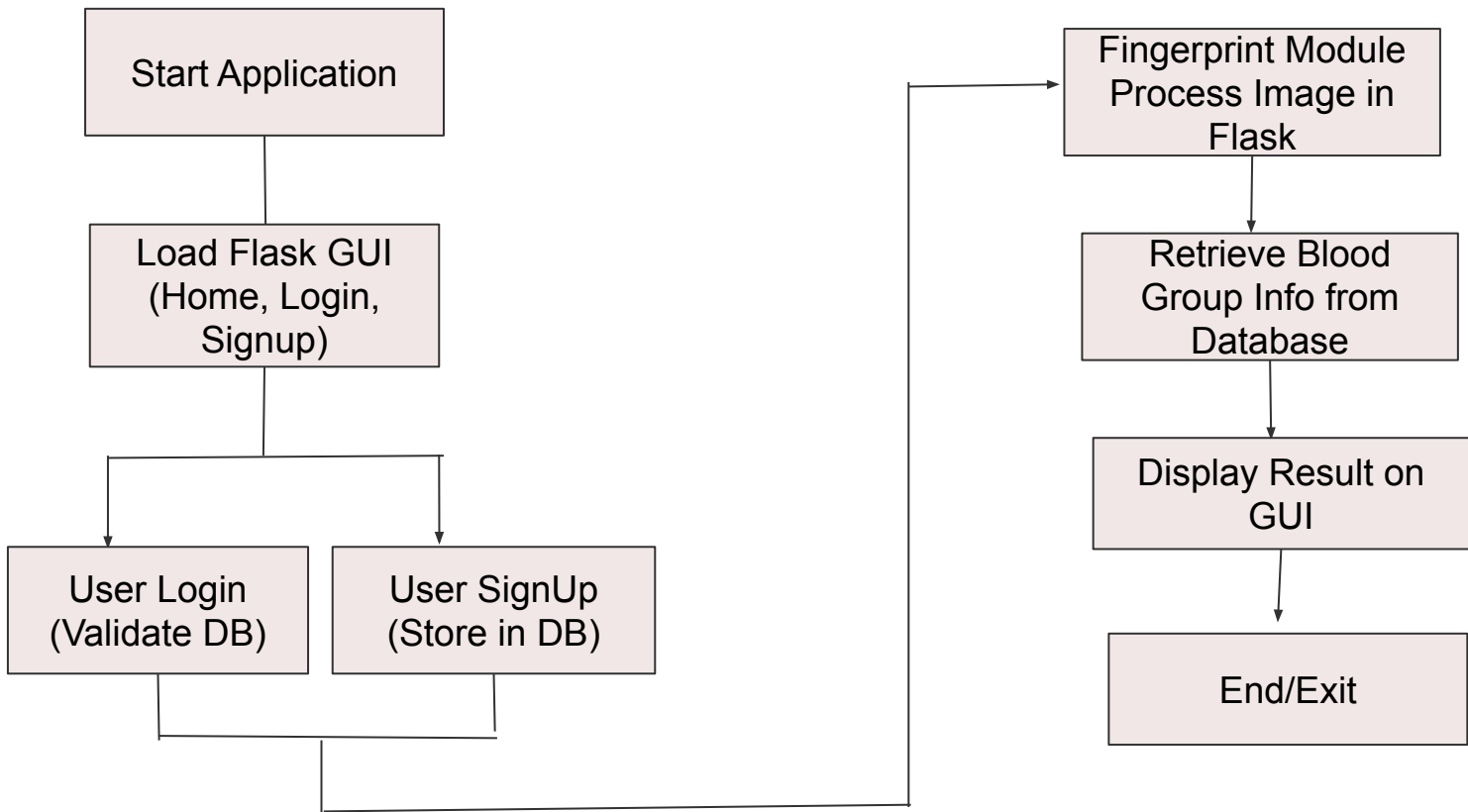
- ✓ **SQLite** → Lightweight, file-based relational database to store user details.

- ◆ **Additional Libraries Used**

- ✓ **OpenCV** (if used) → Helps in fingerprint image preprocessing and feature extraction.

- ✓ **Subprocess** → Enables opening login & signup pages from Flask HTML,CSS GUI.

GUI System Flow



Backend Implementation with Flask

The backend is built using **Flask**, which handles API requests and connects to the database.

◆ How Flask Works in the Project

- 1 User sends data from the GUI (Tkinter) to the backend (Flask API).
- 2 Flask processes the request and interacts with SQLite using SQLAlchemy.
- 3 The response is returned to the GUI, and the result is displayed.

Database Design & Schema

This project uses **SQLite** as the database, which is integrated using Flask-SQLAlchemy.

- ◆ **Database Table: User**

Column Name	Data Type	Constraint
username	String(100)	Not Null
email	String(100)	Unique, Not Null
password	String(100)	Unique, Not Null
Confirm password	String(100)	Not Null

Database Schema

id ^	username	email	password
1	dk	sss@gmail.com	Amadeus1@@
2	adam	adam@gmail.com	12345678
3	bob	bob@outlook.com	qwertyuiop

Login

ken

....

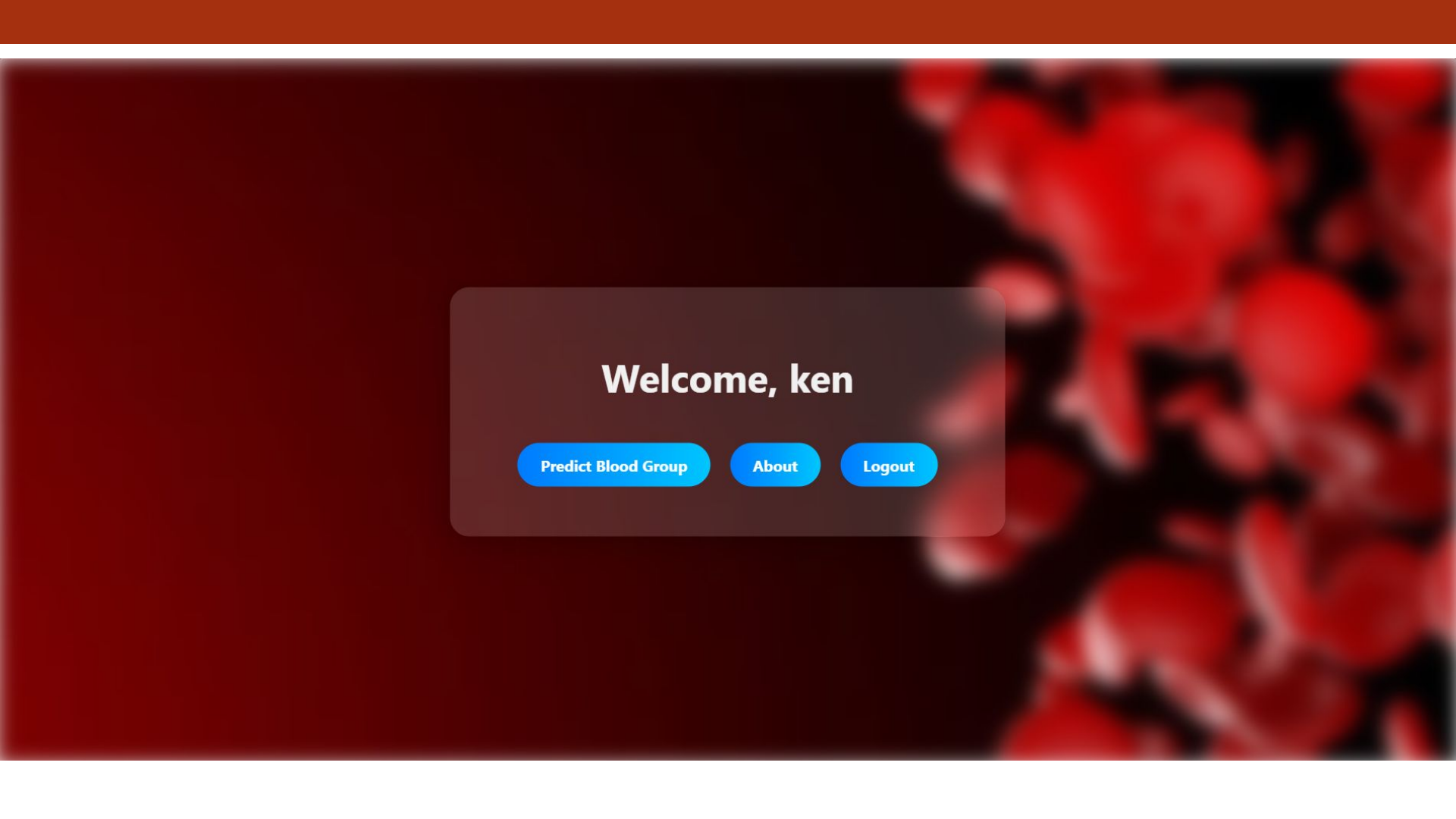
Login

New user? [Sign up here](#)

Create an Account

Sign Up

Already have an account? [Login here](#)

A background image showing a microscopic view of red blood cells. The cells are bright red and spherical, set against a dark, almost black, background. The lighting creates a sense of depth, with some cells appearing more prominent than others.

Welcome, ken

[Predict Blood Group](#)

[About](#)

[Logout](#)

Upload Fingerprint



Choose Fingerprint Image



Predict Blood Group

[← Back to Home](#)



Blood Group Prediction

Predicted Blood Group: A+

Confidence: 99.46%



✂ Handcrafted Features

Edge Count: 11644

Corner Count: 6811

Ridge Thickness: 0.599

Valley Thickness: 0.401

LBP_0: 0.012

LBP_1: 0.036

LBP_2: 0.0201

LBP_3: 0.0007

References

- Mansi K, Hitashree M., Chandana Lakshman Hegdepaper, "Blood Group Determination through Medical Image Processing", International Journal for Research in Applied Science and Engineering Technology. Vol 9, Issue v, May 2021.
- Bansal, R., et al. "Spectroscopic analysis of hand veins for blood group detection using machine learning." Biomedical Signal Processing and Control 58 (2020): 101830.
- Yadav, A., et al. "Blood group detection using deep learning on hyperspectral images of hand veins." Computers in Biology and Medicine 138 (2021): 104825
- Anurag Sadashiv Phad, Tejas Sanjay Targhale, Bharat Bhalshankar, Sunita Kulkurni, "Blood Group Detection by Using Raspberry Pi-3", International Journal of Advanced Research in Electronics and Communication Engineering (IJARECE). Vol 7, Issue 4, [April 2018].